NODE MCU 8266 MODULE

with Arduino we cannot directly connect to internet
so with using node mcu we can directly connect to internet
we can fetch, write data to db

benefits control applinces from anywere of the world, if node is connected to internet

NODE MCU PINS

D0 - D8 -- PWM,
SD3, SD2 -- ONLY GPIO NO PWM

--- FIREBASE ---

READ DATA

Data at a specific node in Firebase RTDB can be read through the following functions: get, getInt, getFloat, getDouble, getBool, getString, getJSON, getArray, getBlob, getFile.

These functions return a boolean value indicating the success of the operation, which will be true if all of the following conditions were met:

Server returns HTTP status code 200
The data types matched between request and response.

The database data’s payload (response) can be read or access through the following Firebase Data object’s functions: fbdo.intData, fbdo.floatData, fbdo.doubleData, fbdo.boolData, fbdo.stringData, fbdo.jsonString, fbdo.jsonObject, fbdo.jsonObjectPtr, fbdo.jsonArray, fbdo.jsonArrayPtr, fbdo.jsonData (for keeping parse/get result), and fbdo.blobData.

If you use a function that doesn’t match the returned data type in the database, it will return empty (string, object, or array).

The data type of the returning payload can be determined by fbdo.getDataType.

The following snippet shows how to get an integer value stored in the test/int node. First, we use the getInt() function; then, we check if the data type is an integer with fbdo.dataType(), and finally, the fdbo.intData() gets the value stored in that node.

```
if (Firebase.RTDB.getInt(&fbdo, "/test/int")) {
  if (fbdo.dataType() == "int") {
    intValue = fbdo.intData();
    Serial.println(intValue);
  }
}
else {
  Serial.println(fbdo.errorReason());
}
```

WRITE DATA

Send Data to the Database
As mentioned in the library documentation, to store data at a specific node in the Firebase RTDB (realtime database), use the following functions: set, setInt, setFloat, setDouble, setString, setJSON, setArray, setBlob, and setFile.

These functions return a boolean value indicating the success of the operation, which will be true if all of the following conditions are met:

Server returns HTTP status code 200.
The data types matched between request and response.

Only setBlob and setFile functions that make a silent request to Firebase server, thus no payload response returned.
In our example, we'll send an integer number, so we need to use the setInt() function as follows:

Firebase.RTDB.setInt(&fbdo, "test/int", count)
The second argument is the database node path, and the last argument is the value you want to pass to that database path—you can choose any other database path. In this case, we're passing the value saved in the count variable.

Here's the complete snippet that stores the value in the database and prints a success or failed message.

```
if (Firebase.RTDB.setInt(&fbdo, "test/int", count)) {
  Serial.println("PASSED");
  Serial.println("PATH: " + fbdo.dataPath());
  Serial.println("TYPE: " + fbdo.dataType());
}
else {
  Serial.println("FAILED");
  Serial.println("REASON: " + fbdo.errorReason());
}
```

We proceed in a similar way to store a float value. We're storing a random float value on the test/float path.

```
// Write an Float number on the database path test/float
if (Firebase.RTDB.setFloat(&fbdo, "test/float", 0.01 + random(0, 100))) {
  Serial.println("PASSED");
  Serial.println("PATH: " + fbdo.dataPath());
  Serial.println("TYPE: " + fbdo.dataType());
}
else {
  Serial.println("FAILED");
  Serial.println("REASON: " + fbdo.errorReason());
}
```

FIREBASE CONNECTIVITY STEPS

---first include all essential lib

```
#if defined(ESP32)
  #include <WiFi.h>
#elif defined(ESP8266)
```

```
  #include <ESP8266WiFi.h>
#endif
#include <Firebase_ESP_Client.h>

//Provide the token generation process info.
#include "addons/TokenHelper.h"
//Provide the RTDB payload printing info and other helper functions.
#include "addons/RTDBHelper.h"

---add your network creds
#define WIFI_SSID "==="
#define WIFI_PASSWORD "---"


---add firebase keys
#define API_KEY "find in project settings "

// Insert RTDB URLefine the RTDB URL */
#define DATABASE_URL "---found in rtdb firebase---"

//Define Firebase Data object
FirebaseData fbdo;

//firebase essential objects
FirebaseAuth auth;
FirebaseConfig config;

---create bool var for signup state
bool signupOk = false;

void setup() {
serial.begin();

connect to wifi -- with ssid, pswd
config firebase -- with api_key, db_url
try to sign up using if else -- if(Firebase.signUp(config,auth,userId, pswd)) {connected} else show error
config token
begin firebase
firebase.reconnectWiFi()


 Serial.begin(115200);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.print("Connecting to Wi-Fi");
  while (WiFi.status() != WL_CONNECTED){
    Serial.print(".");
    delay(300);
  }
  Serial.println();
  Serial.print("Connected with IP: ");
  Serial.println(WiFi.localIP());
  Serial.println();

  /* Assign the api key (required) */
  config.api_key = API_KEY;
```

```
  /* Assign the RTDB URL (required) */
  config.database_url = DATABASE_URL;

  /* Sign up */
  if (Firebase.signUp(&config, &auth, "", "")){
    Serial.println("ok");
    signupOK = true;
  }
  else{
    Serial.printf("%s\n", config.signer.signupError.message.c_str());
  }

  /* Assign the callback function for the long running token generation task */
  config.token_status_callback = tokenStatusCallback; //see addons/TokenHelper.h

  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);

}
```

read, write

firebase ready hai!! && signupOk hai!! give a sligt delay if u want !! -- but wven without delay u are ready to read or write

```
if(Firebase.RTDB.getInt(&fbdo, "location")) { read data eg intValue = fbod.intData();
} else {  fbdo.errorReason()) }
```

getInt(&fbdo, path): Retrieves an integer value from the specified path.
getFloat(&fbdo, path): Retrieves a floating-point value.
getDouble(&fbdo, path): Retrieves a double value.
getBool(&fbdo, path): Retrieves a boolean value.
getString(&fbdo, path): Retrieves a string value.
getJSON(&fbdo, path): Retrieves JSON data as a FirebaseJson object.
getArray(&fbdo, path): Retrieves an array as a FirebaseJsonArray object.
getBlob(&fbdo, path): Retrieves binary data.
getFile(&fbdo, path, filename): Downloads a file from the database to the device storage.
Set/Write Data
setInt(&fbdo, path, value): Sets an integer value at the specified path.
setFloat(&fbdo, path, value): Sets a floating-point value.
setDouble(&fbdo, path, value): Sets a double value.
setBool(&fbdo, path, value): Sets a boolean value.
setString(&fbdo, path, value): Sets a string value.
setJSON(&fbdo, path, FirebaseJson): Sets JSON data from a FirebaseJson object.
setArray(&fbdo, path, FirebaseJsonArray): Sets an array from a FirebaseJsonArray object.
setBlob(&fbdo, path, blob, size): Uploads binary data (blob).
setFile(&fbdo, path, filename): Uploads a file from device storage to the database.
Update Data
updateNode(&fbdo, path, FirebaseJson): Updates specific keys in a node using FirebaseJson.
Delete Data
deleteNode(&fbdo, path): Deletes a node or its content from the database.
Utility Methods
getETag(&fbdo, path): Retrieves the ETag of the node.
getShallowData(&fbdo, path): Retrieves shallow data without downloading the entire node.

backup(&fbdo, path, filename): Backups the data at the specified path to the device.
restore(&fbdo, path, filename): Restores data from a backup file.

ADDING PRIMITIVE FETCHED DATA IN OUR LOCAL VAR
Primitive Data Types

intData(): Retrieves the value as an integer.
Example:

cpp
int intValue = fbdo.intData();
floatData(): Retrieves the value as a float.
Example:

cpp
float floatValue = fbdo.floatData();
doubleData(): Retrieves the value as a double.
Example:

cpp
double doubleValue = fbdo.doubleData();
boolData(): Retrieves the value as a boolean.
Example:

cpp
bool boolValue = fbdo.boolData();
stringData(): Retrieves the value as a string.
Example:

cpp
String stringValue = fbdo.stringData();